

Zero Downtime Deployment: 배포를 운영 전환 절차로 설계하기

무중단 배포는 서버를 바꾸는 기술이 아니라, 변경을 승인·전환·검증·rollback하는 운영 절차다.

요약

GitLab과 Jenkins 기반의 dev·qa·prod 배포 흐름을 versioned bundle, 승인, audit, blue-green, canary, rollback 중심으로 확장했다. MinIO는 배포 산출물의 versioned storage로 사용하고, L7 계층과 연결해 traffic 전환을 통제하는 구조로 정리했다.

핵심 판단은 배포 자동화와 운영 전환을 분리하지 않는 것이다. build가 성공해도 운영 전환은 별도 문제다. 누가 승인했는지, 어떤 bundle이 어느 stage에 적용되었는지, 어떤 traffic이 신규 version으로 이동했는지, 실패 시 어디로 되돌릴지 설명할 수 있어야 한다.

문제 정의

초기 배포 흐름은 GitLab/Jenkins를 중심으로 구성되어 있었다. 개발, QA, 운영 배포가 가능했지만 운영 규모가 커질수록 다음 문제가 생겼다.

- 산출물 version과 실제 운영 적용 상태를 명확히 연결해야 했다.
- 배포 승인과 실행 주체를 audit으로 남겨야 했다.
- 운영 반영 후 일부 고객사에서만 장애가 발생하는 사례가 있었다.
- 전체 rollback과 부분 rollback 기준을 구분해야 했다.
- DB migration은 application rollback과 동일하게 처리할 수 없었다.
- L7 traffic 전환과 deployment 상태가 연결되어야 했다.

책임 경계

계층	책임
GitLab/Jenkins	build, test, artifact 생성, pipeline 실행을 담당한다.
MinIO Bundle Store	versioned artifact와 metadata를 보관한다.
Approval	stage별 승인, diff, 배포 요청, 반려 이력을 남긴다.

계층	책임
Deployment Controller	blue-green, canary, rollback 실행을 관리한다.
OpenResty L7	traffic 전환과 route 단위 차단을 수행한다.
Monitoring	배포 이후 장애, 지연, error rate를 version과 연결한다.

핵심 설계 결정

배포 산출물을 versioned bundle로 다뤘다

운영에서 중요한 것은 “무엇을 배포했는가”를 나중에 재현할 수 있는지다. Jenkins build 결과를 그때그때 소비하는 구조는 추적에 취약하다. MinIO에 versioned bundle과 metadata를 보관하면 어떤 artifact가 어떤 stage에 승인되어 적용되었는지 기록할 수 있다.

```
git commit
-> build
-> test
-> versioned bundle
-> approval
-> stage deploy
-> traffic switch
-> monitoring
-> rollback if needed
```

blue-green과 canary를 L7과 연결했다

Blue-green은 신규 환경을 준비하고 traffic을 전환하는 방식이다. Canary는 일부 traffic만 신규 version으로 보내 검증하는 방식이다. 두 방식 모두 실제 traffic 제어가 필요하므로 L7 계층과 연결되어야 한다.

배포 시스템이 “신규 version이 준비됐다”고 말하는 것과 사용자의 요청이 실제로 어디로 가는지는 다른 문제다. 이 경계를 분리하면 장애 분석이 어려워진다.

DB migration rollback은 별도 문제로 다뤘다

Application rollback은 이전 bundle로 되돌리면 된다. 그러나 DB migration은 항상 되돌릴 수 있는 것이 아니다. schema 변경, data migration, backfill, index 변경은 영향 범위와 시간이 다르다.

따라서 DB 변경은 application 배포와 같은 버튼으로 단순화하지 않았다. expand/contract, backward compatibility, dry-run, 검증 query, manual approval 기준을 별도로 뒀다.

버린 선택과 이유

Jenkins job 성공을 배포 완료로 보는 방식은 제외했다. build가 성공해도 운영 적용은 별개의 문제다. 어떤 bundle이 어느 stage에 반영되었는지, 누가 승인했는지, canary가 어떤 기준으로 진행되었는지 남지 않으면 rollback도 불안정해진다.

서버에 직접 산출물을 복사하는 방식도 장기 운영에는 맞지 않았다. 동일한 산출물을 재현할 수 있어야 하고, 승인된 bundle과 실제 배포된 bundle이 같다는 증거가 필요했다. 그래서 versioned bundle 저장소가 필요했다.

DB migration을 application rollback과 같은 절차로 묶는 방식도 피했다. application은 이전 version으로 빠르게 되돌릴 수 있지만, DB schema와 data migration은 비가역 변경을 포함할 수 있다. 따라서 migration은 expand/contract, backward compatibility, dry-run 기준을 별도로 뒀다.

배포에서 남겨야 하는 증거

증거	목적
bundle hash	승인된 산출물과 배포된 산출물이 같은지 확인한다.
approval record	배포 승인자, 실행자, 사유를 남긴다.
traffic switch log	blue/green 또는 canary 전환 시점을 재현한다.
health metric snapshot	전환 판단 근거를 남긴다.
rollback target	실패 시 돌아갈 version을 명확히 한다.

운영 시나리오

Canary 장애

신규 version을 일부 고객사나 일부 traffic에만 적용한다. error rate, latency, 특정 업무 오류가 증가하면 L7에서 canary traffic을 중단하고 이전 version으로 전환한다. 이때 장애 이벤트는 bundle version, route, tenant, stage와 연결되어야 한다.

Emergency rollback

운영 반영 후 핵심 업무 장애가 발생하면 신규 build를 기다리지 않고 이전 승인 bundle로 되돌려야 한다. rollback은 빠르게 실행되어야 하지만, 누가 어떤 사유로 어떤 version으로 되돌렸는지 audit으로 남아야 한다.

Partial block

특정 route나 고객사에서만 문제가 발생하면 전체 rollback보다 route block이나 tenant-level rollback이 더 안전할 수 있다. 이 판단을 위해 L7 정책, 배포 version, 장애 이벤트가 연결되어야 한다.

운영 검증

검증 기준은 다음과 같았다.

- bundle version과 운영 적용 상태가 일치하는가.
- 승인자, 실행자, 변경 diff, 적용 stage가 audit으로 남는가.
- canary traffic이 의도한 기준으로만 이동하는가.
- 배포 이후 장애 이벤트가 version과 연결되는가.
- rollback 대상 version이 명확하고 재현 가능한가.
- DB migration은 application rollback과 독립적으로 검증되는가.

역할과 책임

GitLab/Jenkins 기반 배포 흐름의 운영 한계를 분석하고, MinIO versioned bundle, approval, audit, blue-green/canary, L7 traffic switch, rollback, DB migration 분리 기준을 설계했다.

설계 결론

배포는 pipeline 성공으로 끝나지 않는다. 운영 전환은 승인, traffic 제어, 관측, rollback을 포함하는 절차다. 무중단 배포의 본질은 중단 없이 서버를 바꾸는 것이 아니라, 변경을 안전하게 적용하고 실패 시 되돌릴 수 있는 운영 구조를 갖추는 것이다.